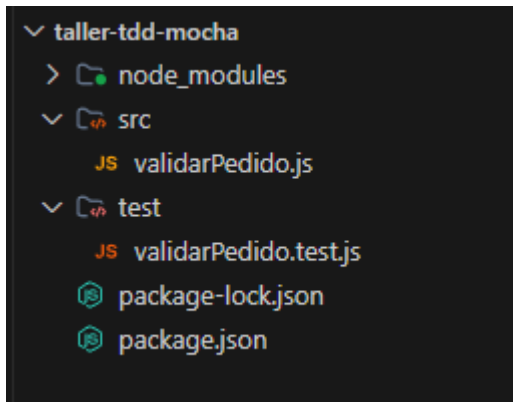


Nombre: Ismael Peñaloza

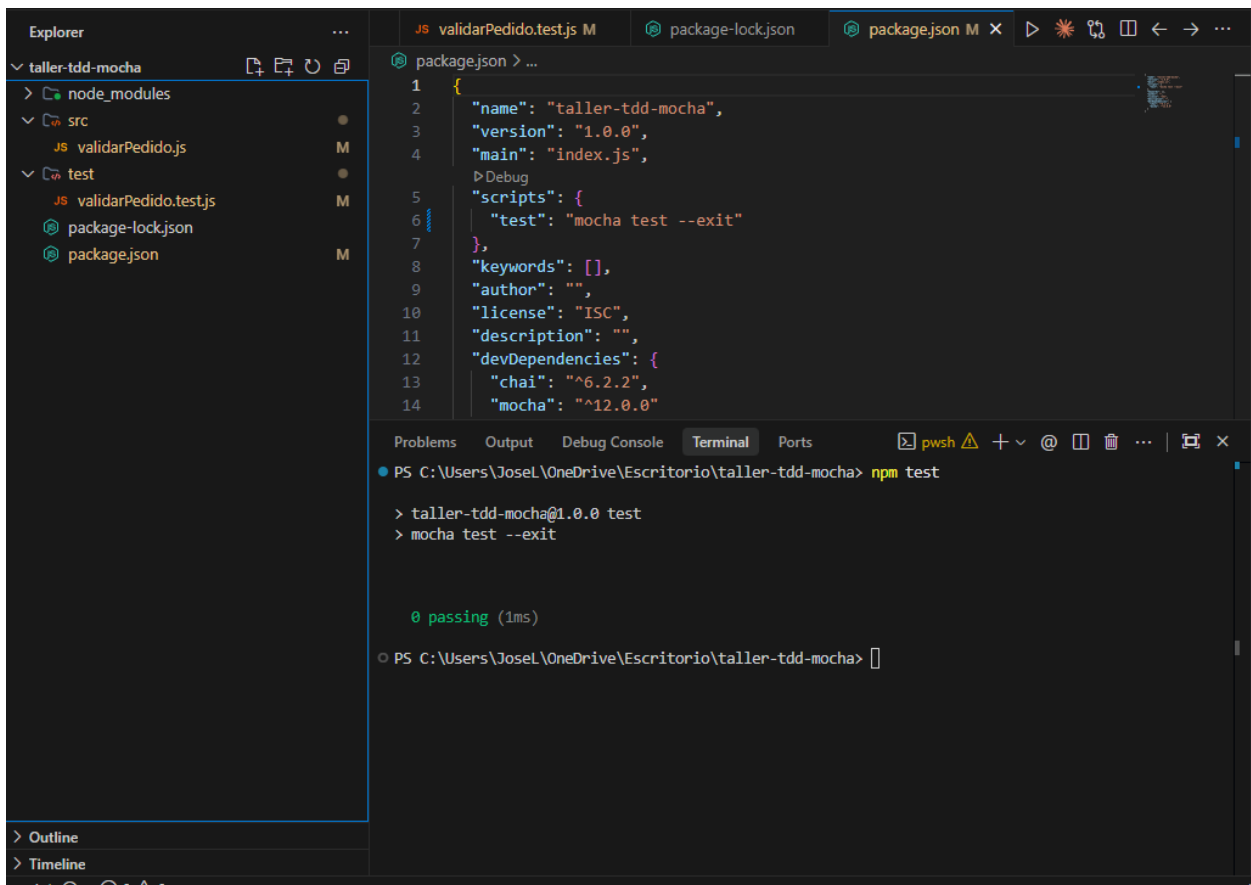
Docente: Amilkar Hernandez

Taller Pruebas al Back

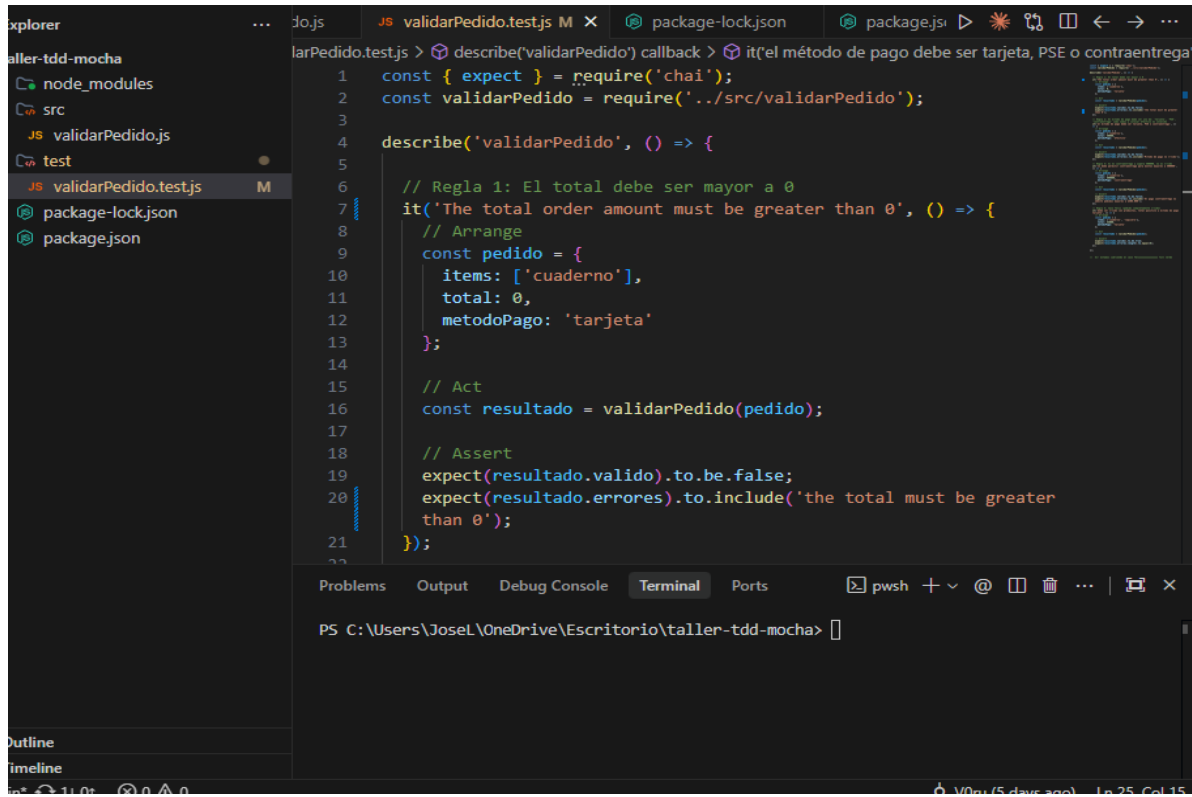
Creacion del entorno



Primer Test (Solo con la creación del entorno)



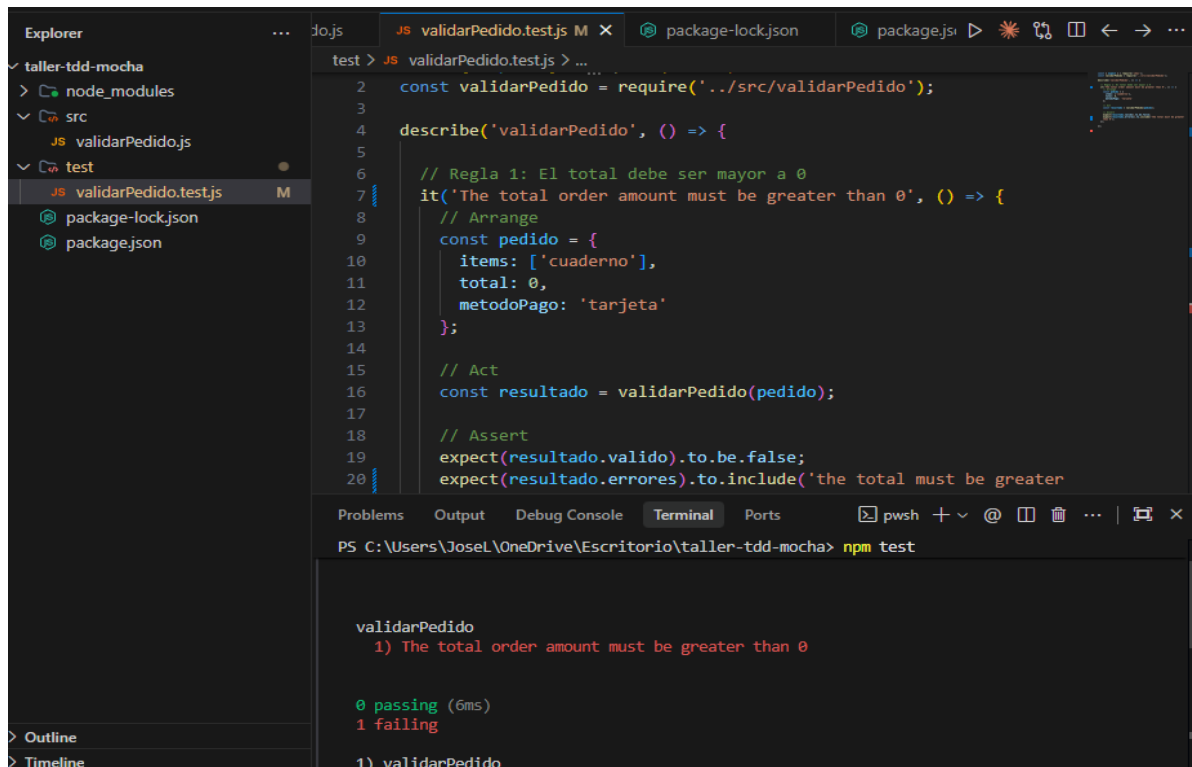
Regla 1 **RED** (El total debe ser mayor a 0)



```
1 const { expect } = require('chai');
2 const validarPedido = require('../src/validarPedido');
3
4 describe('validarPedido', () => {
5
6   // Regla 1: El total debe ser mayor a 0
7   it('The total order amount must be greater than 0', () => {
8     // Arrange
9     const pedido = {
10       items: ['cuaderno'],
11       total: 0,
12       metodoPago: 'tarjeta'
13     };
14
15     // Act
16     const resultado = validarPedido(pedido);
17
18     // Assert
19     expect(resultado.valido).to.be.false;
20     expect(resultado.errores).to.include('the total must be greater than 0');
21   });
22 });
```

Terminal output:

```
PS C:\Users\JoseL\OneDrive\Escritorio\taller-tdd-mocha>
```



```
2 const validarPedido = require('../src/validarPedido');
3
4 describe('validarPedido', () => {
5
6   // Regla 1: El total debe ser mayor a 0
7   it('The total order amount must be greater than 0', () => {
8     // Arrange
9     const pedido = {
10       items: ['cuaderno'],
11       total: 0,
12       metodoPago: 'tarjeta'
13     };
14
15     // Act
16     const resultado = validarPedido(pedido);
17
18     // Assert
19     expect(resultado.valido).to.be.false;
20     expect(resultado.errores).to.include('the total must be greater than 0');
21   });
22 });
```

Terminal output:

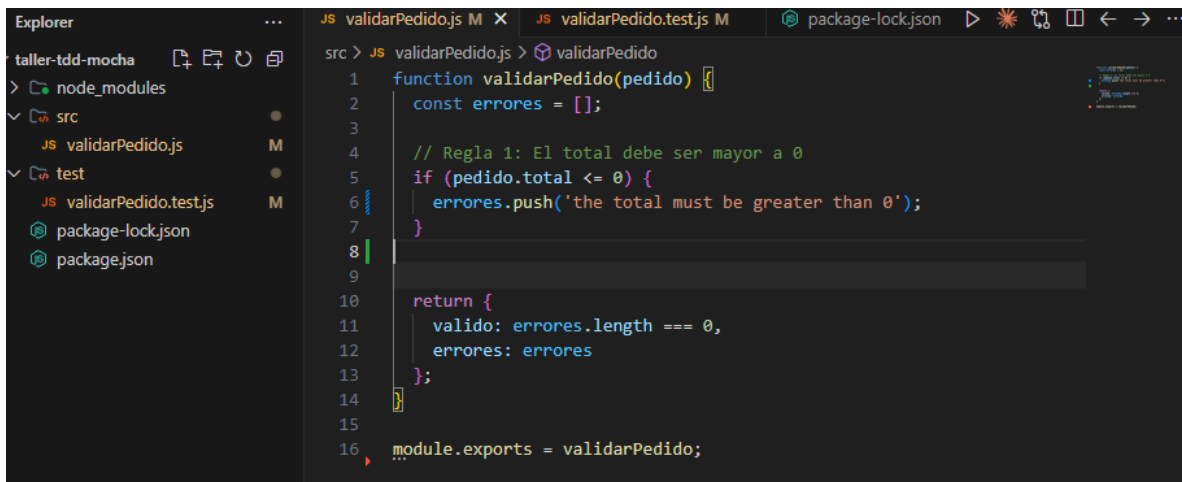
```
test > npm test

validarPedido
  1) The total order amount must be greater than 0

0 passing (6ms)
1 failing

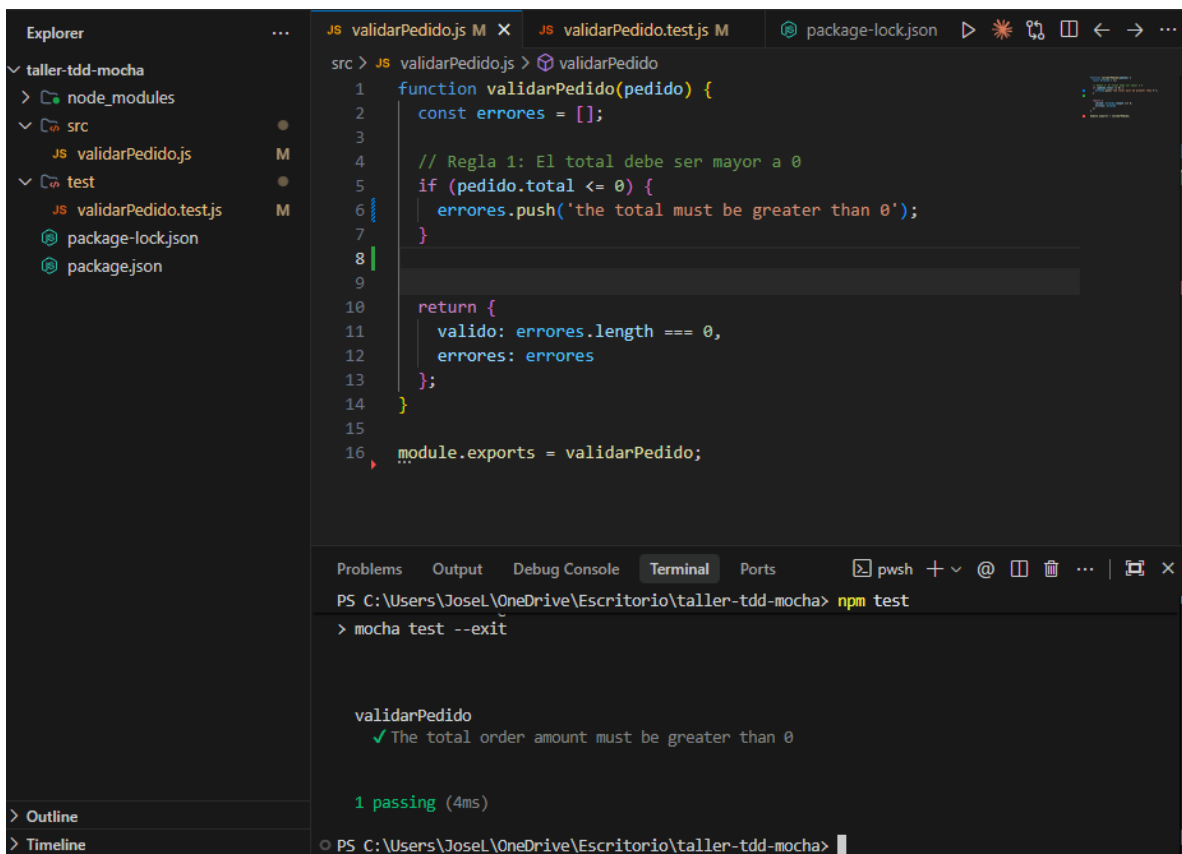
1) validarPedido
```

Regla 1 GREEN



The screenshot shows the VS Code editor with the file explorer on the left. The 'src' folder contains 'validarPedido.js' and 'validarPedido.test.js'. The 'test' folder contains 'validarPedido.test.js'. The main editor shows the 'validarPedido.js' file with the following code:

```
1 function validarPedido(pedido) {  
2   const errores = [];  
3  
4   // Regla 1: El total debe ser mayor a 0  
5   if (pedido.total <= 0) {  
6     errores.push('the total must be greater than 0');  
7   }  
8  
9  
10  return {  
11    valido: errores.length === 0,  
12    errores: errores  
13  };  
14 }  
15  
16 module.exports = validarPedido;
```



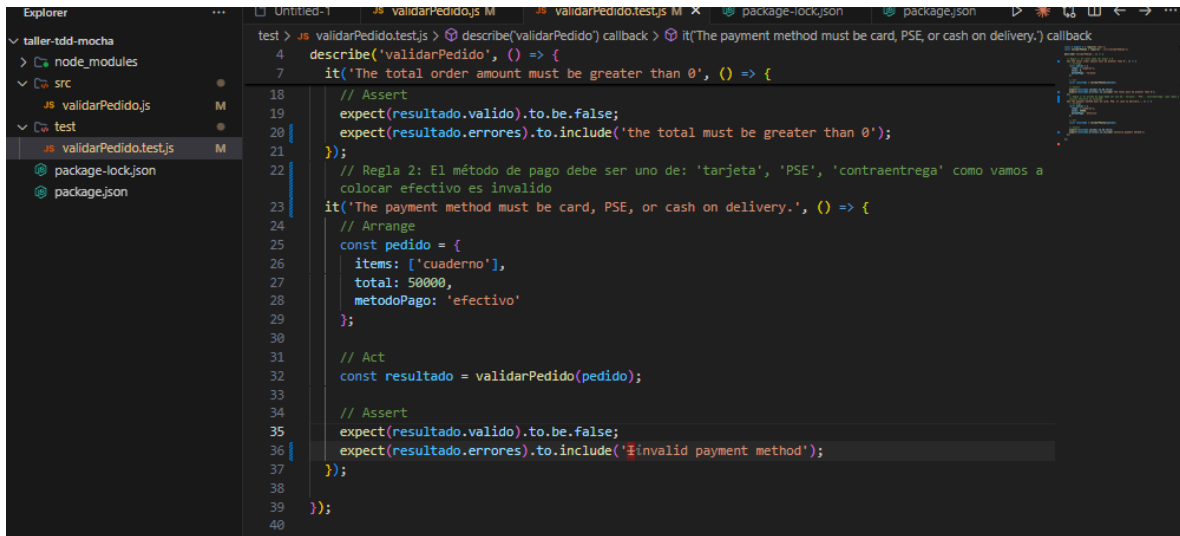
The screenshot shows the VS Code editor with the file explorer on the left. The 'src' folder contains 'validarPedido.js' and 'validarPedido.test.js'. The 'test' folder contains 'validarPedido.test.js'. The main editor shows the 'validarPedido.js' file with the following code:

```
1 function validarPedido(pedido) {  
2   const errores = [];  
3  
4   // Regla 1: El total debe ser mayor a 0  
5   if (pedido.total <= 0) {  
6     errores.push('the total must be greater than 0');  
7   }  
8  
9  
10  return {  
11    valido: errores.length === 0,  
12    errores: errores  
13  };  
14 }  
15  
16 module.exports = validarPedido;
```

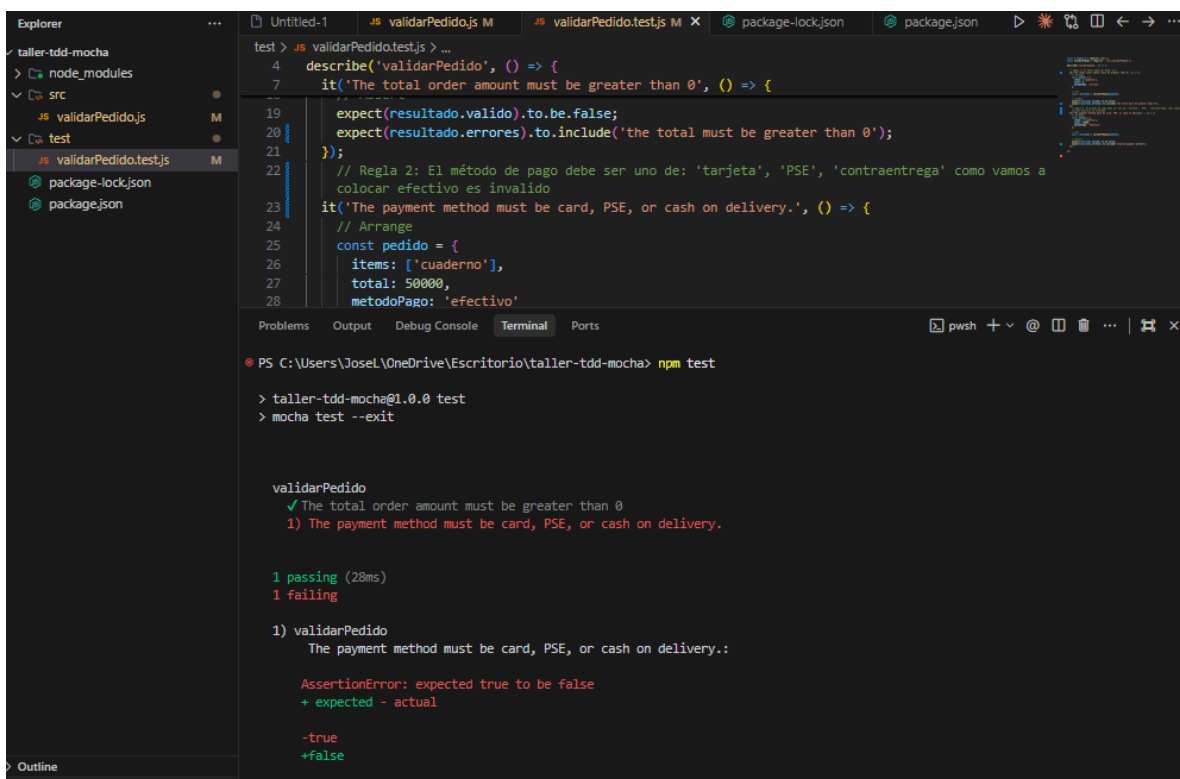
The terminal at the bottom shows the command 'npm test' and the output:

```
PS C:\Users\JoseL\OneDrive\Escritorio\taller-tdd-mocha> npm test  
> mocha test --exit  
  
validarPedido  
  ✓ The total order amount must be greater than 0  
  
1 passing (4ms)
```

Regla 2 **RED** (El método de pago debe ser uno de: 'tarjeta', 'PSE', 'contraentrega' como vamos a colocar efectivo es invalido)



```
test > JS validarPedido.test.js > describe('validarPedido') callback > it('The payment method must be card, PSE, or cash on delivery.') callback
4 describe('validarPedido', () => {
7   it('The total order amount must be greater than 0', () => {
18     // Assert
19     expect(resultado.valido).to.be.false;
20     expect(resultado.errores).to.include('the total must be greater than 0');
21   });
22   // Regla 2: El método de pago debe ser uno de: 'tarjeta', 'PSE', 'contraentrega' como vamos a
23   // colocar efectivo es invalido
24   it('The payment method must be card, PSE, or cash on delivery.', () => {
25     // Arrange
26     const pedido = {
27       items: ['cuaderno'],
28       total: 50000,
29       metodoPago: 'efectivo'
30     };
31     // Act
32     const resultado = validarPedido(pedido);
33     // Assert
34     expect(resultado.valido).to.be.false;
35     expect(resultado.errores).to.include('invalid payment method');
36   });
37 }
38 }
39 }
40 }
```



```
test > JS validarPedido.test.js > ...
4 describe('validarPedido', () => {
7   it('The total order amount must be greater than 0', () => {
19     expect(resultado.valido).to.be.false;
20     expect(resultado.errores).to.include('the total must be greater than 0');
21   });
22   // Regla 2: El método de pago debe ser uno de: 'tarjeta', 'PSE', 'contraentrega' como vamos a
23   // colocar efectivo es invalido
24   it('The payment method must be card, PSE, or cash on delivery.', () => {
25     // Arrange
26     const pedido = {
27       items: ['cuaderno'],
28       total: 50000,
29       metodoPago: 'efectivo'
30     };
31     // Act
32     const resultado = validarPedido(pedido);
33     // Assert
34     expect(resultado.valido).to.be.false;
35     expect(resultado.errores).to.include('invalid payment method');
36   });
37 }
38 }
39 }
40 }
```

```
PS C:\Users\JoseL\OneDrive\Escritorio\taller-tdd-mocha> npm test

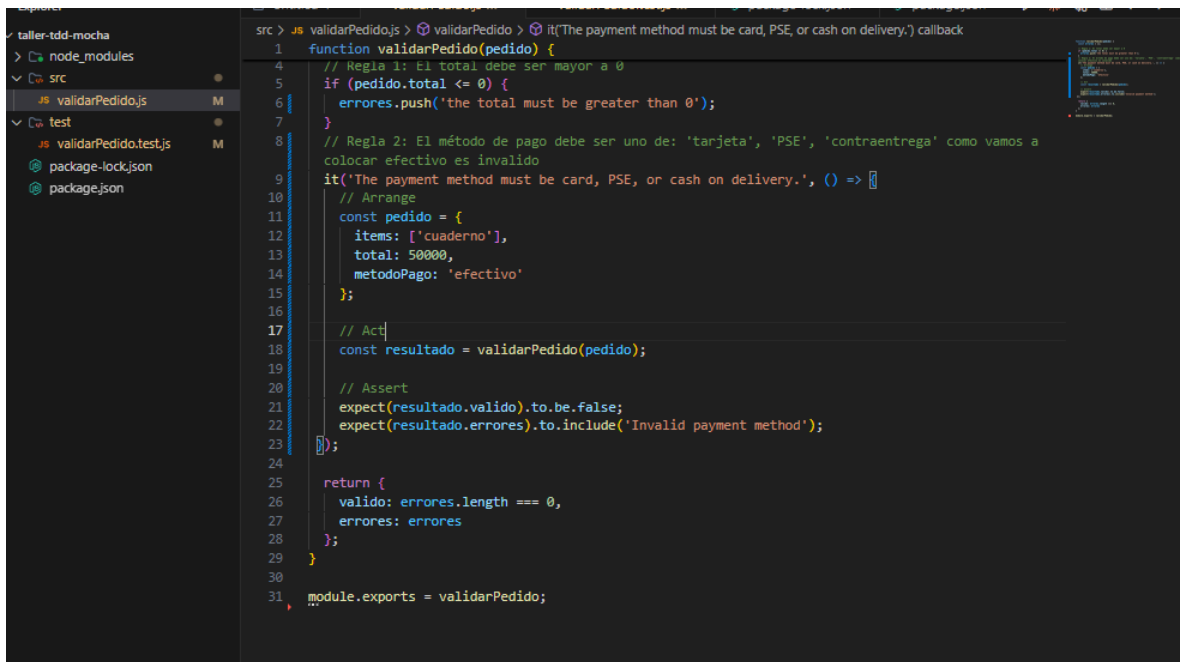
> taller-tdd-mocha@1.0.0 test
> mocha test --exit

validarPedido
  ✓ The total order amount must be greater than 0
  1) The payment method must be card, PSE, or cash on delivery.

1 passing (28ms)
1 failing

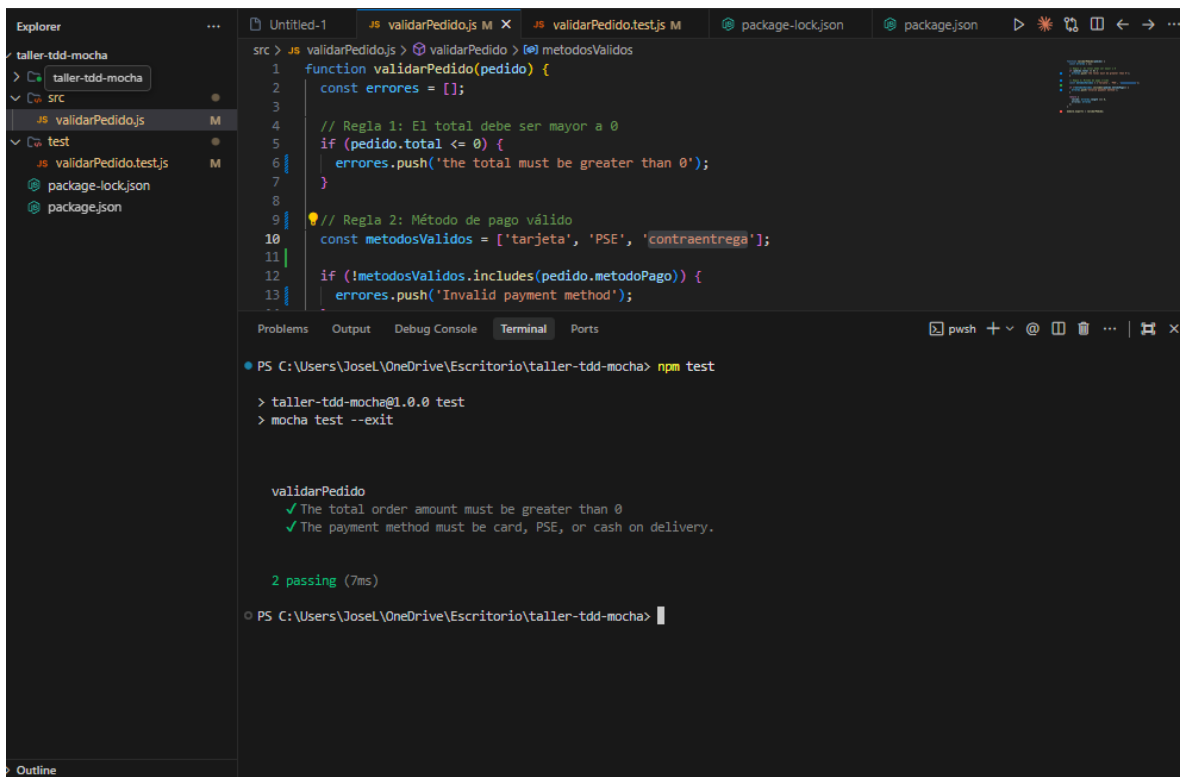
1) validarPedido
   The payment method must be card, PSE, or cash on delivery.:
     AssertionError: expected true to be false
     + expected - actual
     -true
     +false
```

Regla 2 Green



The screenshot shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'taller-tdd-mocha' with a 'src' directory containing 'validarPedido.js' and 'validarPedido.test.js'. The code editor shows the implementation of the 'validarPedido' function in 'validarPedido.js'. The function takes a 'pedido' object and returns an object with 'valido' and 'errores' properties. It implements two rules: Rule 1 (total must be greater than 0) and Rule 2 (payment method must be card, PSE, or cash on delivery). The function uses 'expect' and 'it' from Jest for testing.

```
src > JS validarPedido.js > validarPedido > it('The payment method must be card, PSE, or cash on delivery.') callback
1 function validarPedido(pedido) {
2   // Regla 1: El total debe ser mayor a 0
3   if (pedido.total <= 0) {
4     errores.push('the total must be greater than 0');
5   }
6   // Regla 2: El método de pago debe ser uno de: 'tarjeta', 'PSE', 'contraentrega' como vamos a
7   // colocar efectivo es invalido
8   it('The payment method must be card, PSE, or cash on delivery.', () => {
9     // Arrange
10    const pedido = {
11      items: ['cuaderno'],
12      total: 50000,
13      metodoPago: 'efectivo'
14    };
15    // Act
16    const resultado = validarPedido(pedido);
17    // Assert
18    expect(resultado.valido).to.be.false;
19    expect(resultado.errores).to.include('Invalid payment method');
20  });
21  return {
22    valido: errores.length === 0,
23    errores: errores
24  };
25 }
26 module.exports = validarPedido;
```



The screenshot shows the same VS Code editor with the 'validarPedido.test.js' file open. The test results are displayed in the 'Terminal' tab at the bottom. The tests pass, indicating that the implementation is correct.

```
src > JS validarPedido.js > validarPedido > it('The payment method must be card, PSE, or cash on delivery.') callback
1 function validarPedido(pedido) {
2   const errores = [];
3   // Regla 1: El total debe ser mayor a 0
4   if (pedido.total <= 0) {
5     errores.push('the total must be greater than 0');
6   }
7   // Regla 2: Método de pago válido
8   const metodosValidos = ['tarjeta', 'PSE', 'contraentrega'];
9   if (!metodosValidos.includes(pedido.metodoPago)) {
10    errores.push('Invalid payment method');
11  }
12 }
13
```

terminal

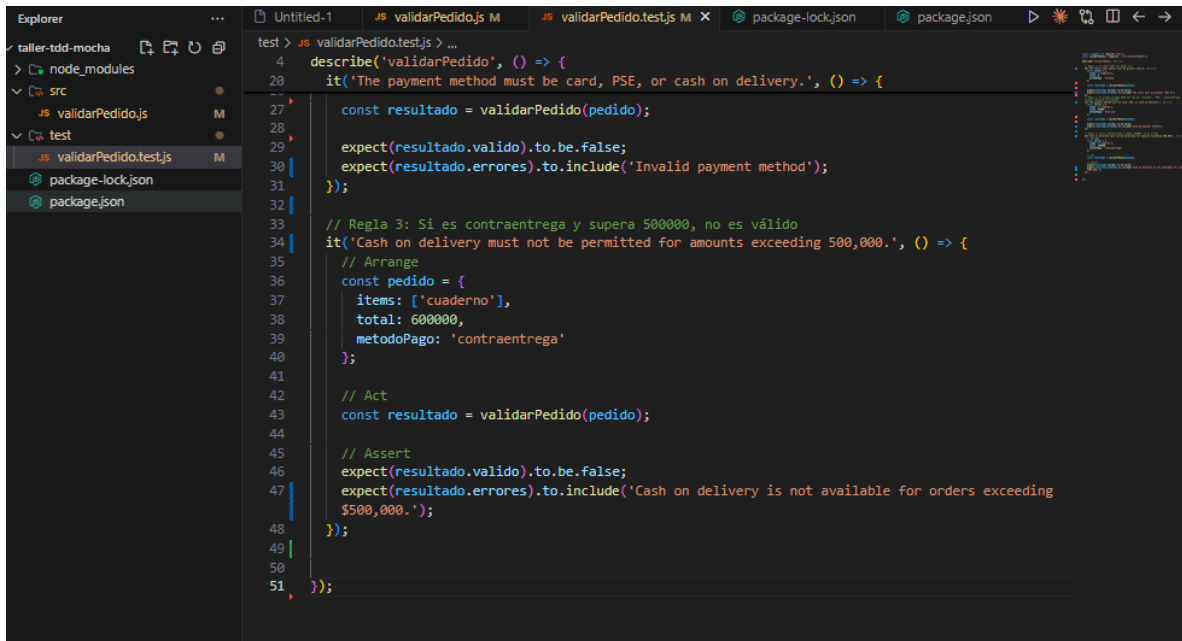
```
PS C:\Users\JoseL\OneDrive\Escritorio\taller-tdd-mocha> npm test
> taller-tdd-mocha@1.0.0 test
> mocha test --exit

validarPedido
  ✓ The total order amount must be greater than 0
  ✓ The payment method must be card, PSE, or cash on delivery.

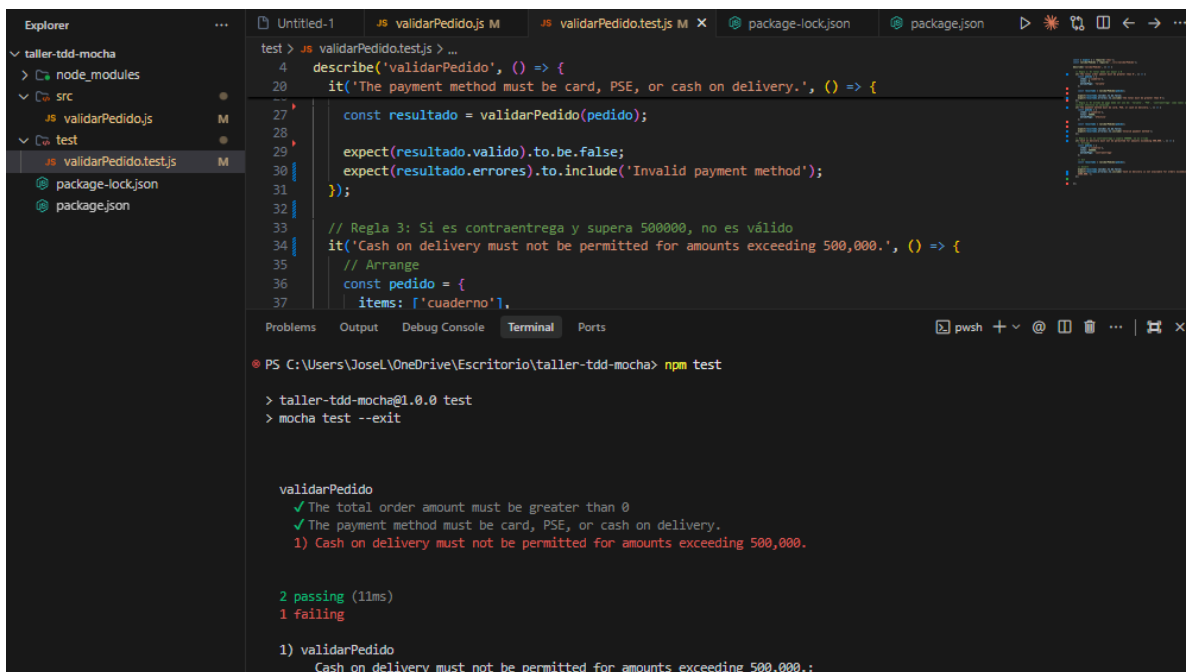
2 passing (7ms)

PS C:\Users\JoseL\OneDrive\Escritorio\taller-tdd-mocha>
```

Regla 3 **RED** (Si es contraentrega y supera 500000, no es válido)



```
test > .\src\validarPedido.test.js > ...
4 describe('validarPedido', () => {
20   it('The payment method must be card, PSE, or cash on delivery.', () => {
27     const resultado = validarPedido(pedido);
28
29     expect(resultado.valido).to.be.false;
30     expect(resultado.errores).to.include('Invalid payment method');
31   });
32
33   // Regla 3: Si es contraentrega y supera 500000, no es válido
34   it('Cash on delivery must not be permitted for amounts exceeding 500,000.', () => {
35     // Arrange
36     const pedido = {
37       items: ['cuaderno'],
38       total: 600000,
39       metodoPago: 'contraentrega'
40     };
41
42     // Act
43     const resultado = validarPedido(pedido);
44
45     // Assert
46     expect(resultado.valido).to.be.false;
47     expect(resultado.errores).to.include('Cash on delivery is not available for orders exceeding $500,000.');
```



```
test > .\src\validarPedido.test.js > ...
4 describe('validarPedido', () => {
20   it('The payment method must be card, PSE, or cash on delivery.', () => {
27     const resultado = validarPedido(pedido);
28
29     expect(resultado.valido).to.be.false;
30     expect(resultado.errores).to.include('Invalid payment method');
31   });
32
33   // Regla 3: Si es contraentrega y supera 500000, no es válido
34   it('Cash on delivery must not be permitted for amounts exceeding 500,000.', () => {
35     // Arrange
36     const pedido = {
37       items: ['cuaderno'],
38       total: 600000,
39       metodoPago: 'contraentrega'
40     };
41
42     // Act
43     const resultado = validarPedido(pedido);
44
45     // Assert
46     expect(resultado.valido).to.be.false;
47     expect(resultado.errores).to.include('Cash on delivery is not available for orders exceeding $500,000.');
```

Problems Output Debug Console Terminal Ports

PS C:\Users\JoseL\OneDrive\Escritorio\taller-tdd-mocha> npm test

> taller-tdd-mocha@1.0.0 test

> mocha test --exit

validarPedido

- ✓ The total order amount must be greater than 0
- ✓ The payment method must be card, PSE, or cash on delivery.
- 1) Cash on delivery must not be permitted for amounts exceeding 500,000.

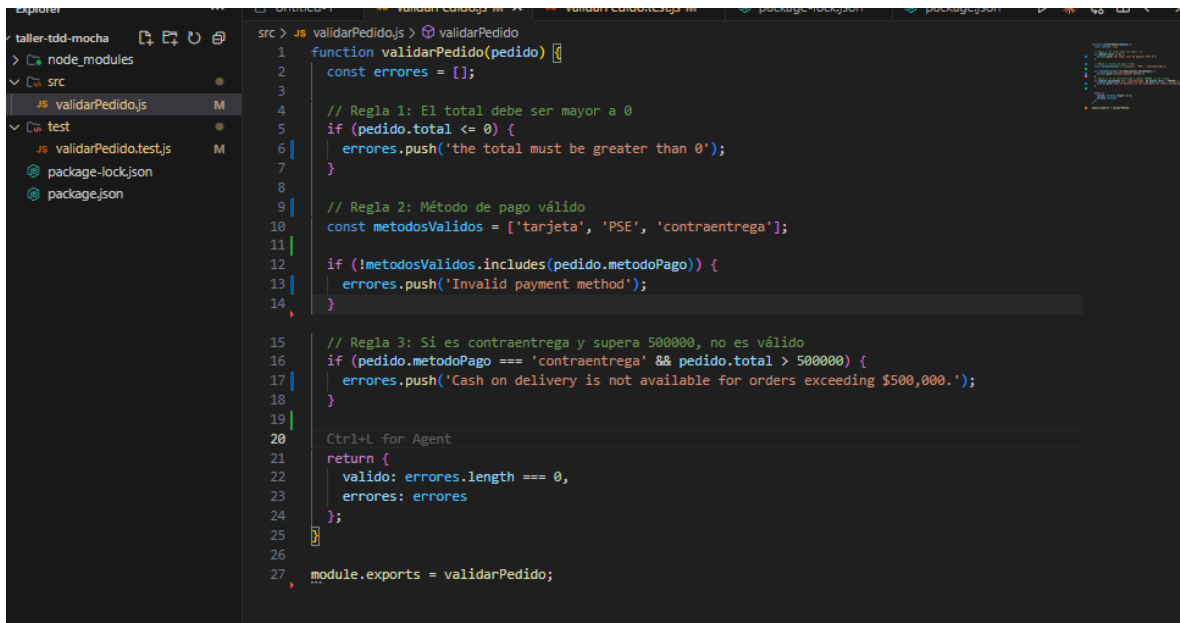
2 passing (11ms)

1 failing

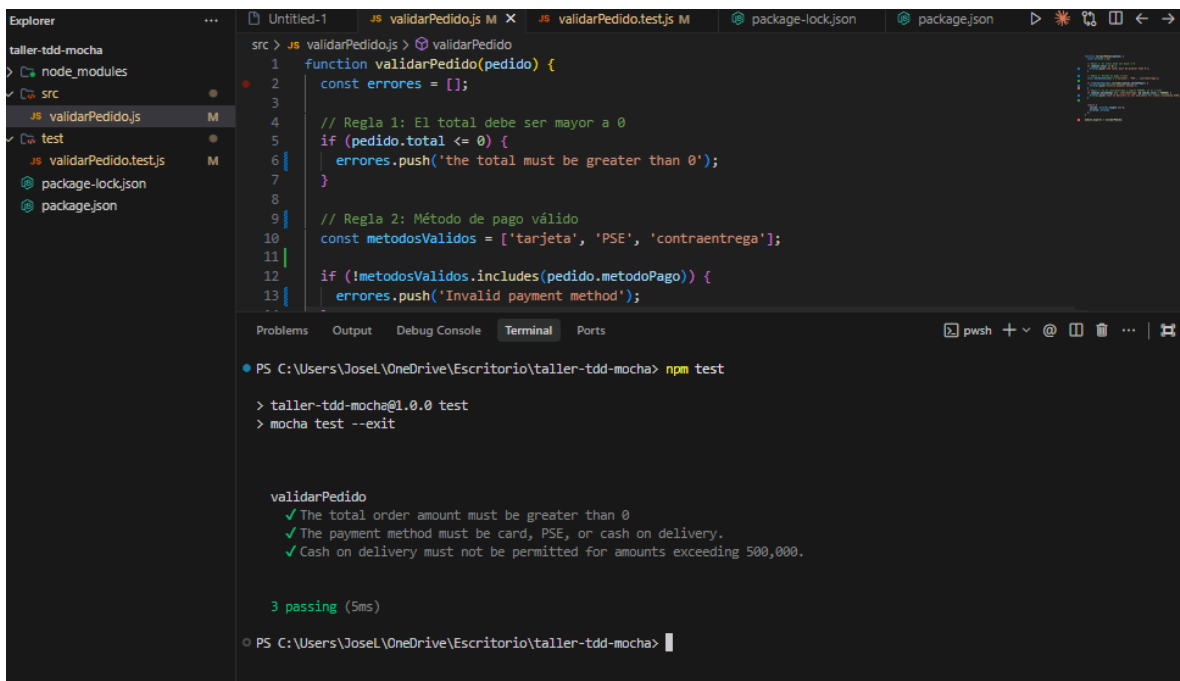
1) validarPedido

Cash on delivery must not be permitted for amounts exceeding 500,000.:

Regla 3 GREEN



```
src > JS validarPedido.js > validarPedido
1 function validarPedido(pedido) {
2   const errores = [];
3
4   // Regla 1: El total debe ser mayor a 0
5   if (pedido.total <= 0) {
6     errores.push('the total must be greater than 0');
7   }
8
9   // Regla 2: Método de pago válido
10  const metodosValidos = ['tarjeta', 'PSE', 'contraentrega'];
11
12  if (!metodosValidos.includes(pedido.metodoPago)) {
13    errores.push('Invalid payment method');
14  }
15
16  // Regla 3: Si es contraentrega y supera 500000, no es válido
17  if (pedido.metodoPago === 'contraentrega' && pedido.total > 500000) {
18    errores.push('Cash on delivery is not available for orders exceeding $500,000.');
```



```
src > JS validarPedido.js > validarPedido
1 function validarPedido(pedido) {
2   const errores = [];
3
4   // Regla 1: El total debe ser mayor a 0
5   if (pedido.total <= 0) {
6     errores.push('the total must be greater than 0');
7   }
8
9   // Regla 2: Método de pago válido
10  const metodosValidos = ['tarjeta', 'PSE', 'contraentrega'];
11
12  if (!metodosValidos.includes(pedido.metodoPago)) {
13    errores.push('Invalid payment method');
```

Problems Output Debug Console Terminal Ports

PS C:\Users\Josel\OneDrive\Escritorio\taller-tdd-mocha> npm test

> taller-tdd-mocha@1.0.0 test

> mocha test --exit

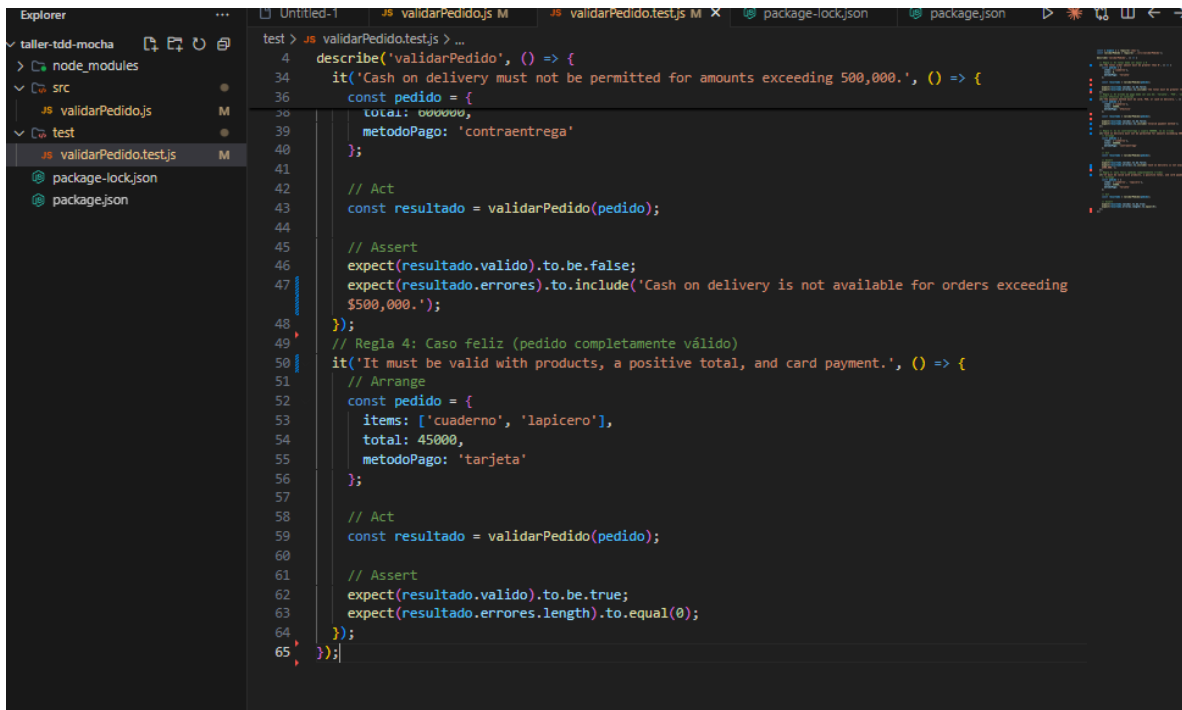
validarPedido

- ✓ The total order amount must be greater than 0
- ✓ The payment method must be card, PSE, or cash on delivery.
- ✓ Cash on delivery must not be permitted for amounts exceeding 500,000.

3 passing (5ms)

PS C:\Users\Josel\OneDrive\Escritorio\taller-tdd-mocha>

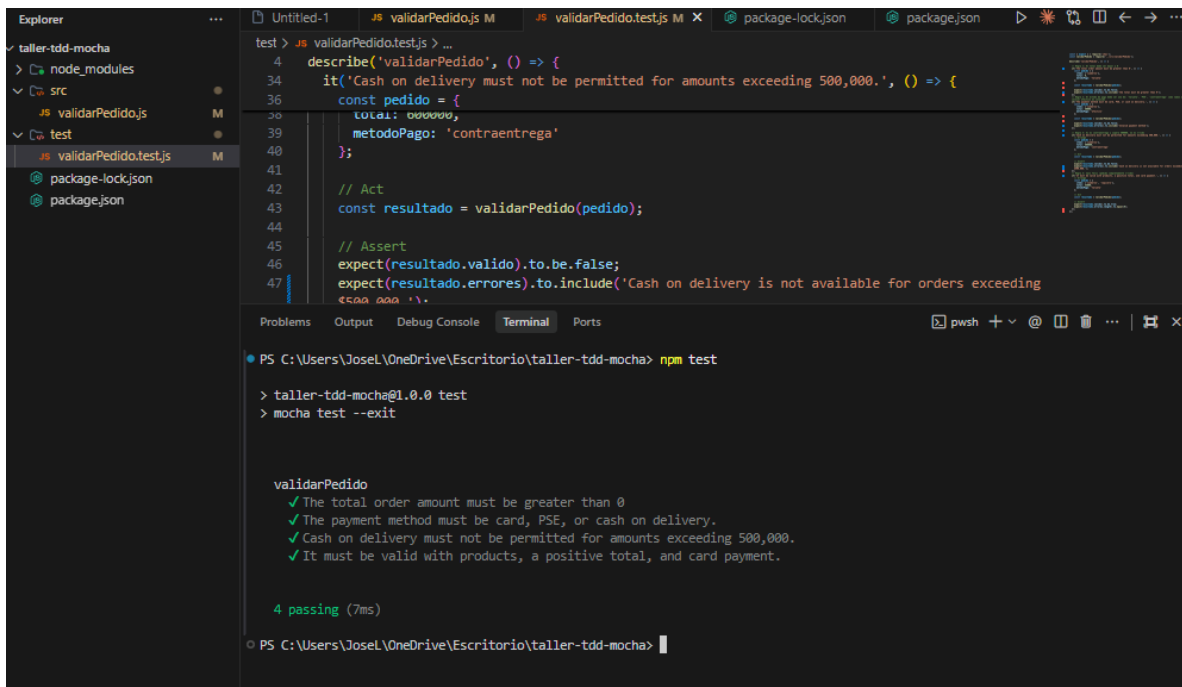
CAMINO FELIZ



The screenshot shows a VS Code editor with the following files in the Explorer: `taller-tdd-mocha`, `node_modules`, `src` (containing `validarPedido.js`), and `test` (containing `validarPedido.test.js`, `package-lock.json`, and `package.json`). The `validarPedido.test.js` file is open, showing the following code:

```
test > JS validarPedido.test.js > ...
4 describe('validarPedido', () => {
34   it('Cash on delivery must not be permitted for amounts exceeding 500,000.', () => {
36     const pedido = {
38       total: 600000,
39       metodoPago: 'contraentrega'
40     };
41
42     // Act
43     const resultado = validarPedido(pedido);
44
45     // Assert
46     expect(resultado.valido).to.be.false;
47     expect(resultado.errores).to.include('Cash on delivery is not available for orders exceeding $500,000.');
```

```
48   });
49   // Regla 4: Caso feliz (pedido completamente válido)
50   it('It must be valid with products, a positive total, and card payment.', () => {
51     // Arrange
52     const pedido = {
53       items: ['cuaderno', 'lapicero'],
54       total: 45000,
55       metodoPago: 'tarjeta'
56     };
57
58     // Act
59     const resultado = validarPedido(pedido);
60
61     // Assert
62     expect(resultado.valido).to.be.true;
63     expect(resultado.errores.length).to.equal(0);
64   });
65 });
```



The screenshot shows the same VS Code editor as above, but with the terminal window open at the bottom. The terminal shows the command `npm test` being executed, which runs the tests defined in `validarPedido.test.js`. The output of the tests is as follows:

```
PS C:\Users\Josef\OneDrive\Escritorio\taller-tdd-mocha> npm test

> taller-tdd-mocha@1.0.0 test
> mocha test --exit

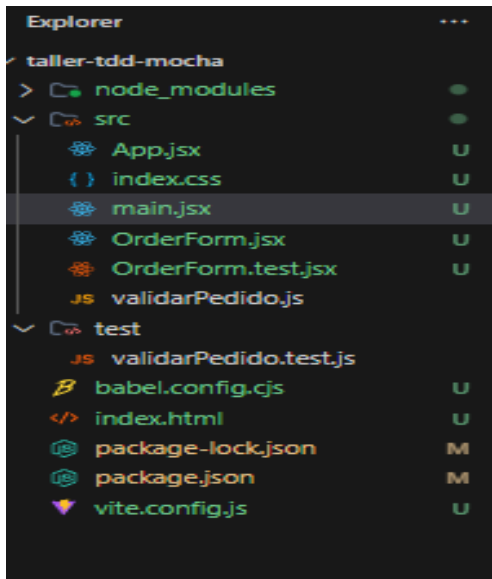
validarPedido
  ✓ The total order amount must be greater than 0
  ✓ The payment method must be card, PSE, or cash on delivery.
  ✓ Cash on delivery must not be permitted for amounts exceeding 500,000.
  ✓ It must be valid with products, a positive total, and card payment.

4 passing (7ms)
```

The terminal also shows the command `PS C:\Users\Josef\OneDrive\Escritorio\taller-tdd-mocha>` at the bottom.

Bueno ahora vamos a realizar las pruebas al front

CREACION DEL ENTORNO Y DEPENDENCIAS:



```
{
  "name": "taller-front-tdd",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "test": "jest",
    "test:coverage": "jest --coverage",
    "test:backend": "mocha test --exit"
  },
  "jest": {
    "testEnvironment": "jsdom",
    "setupFilesAfterEnv": [
      "@testing-library/jest-dom"
    ],
    "moduleNameMapper": {
      "\\.(css|less|scss)$": "identity-obj-proxy"
    },
    "transform": {
      "^.+\\.jsx?$": "babel-jest"
    },
    "coverageReporters": [
      "text",
      "html"
    ],
    "collectCoverageFrom": [
      "src/**/*.js",
      "!src/main.jsx",
      "!src/validarPedido.js"
    ]
  },
  "dependencies": {
    "react": "^19.0.0",
    "react-dom": "^19.0.0"
  }
}
```

```
},
"devDependencies": {
  "@babel/core": "^7.26.0",
  "@babel/preset-env": "^7.26.0",
  "@babel/preset-react": "^7.26.0",
  "@testing-library/dom": "^10.4.0",
  "@testing-library/jest-dom": "^6.6.3",
  "@testing-library/react": "^16.2.0",
  "@testing-library/user-event": "^14.6.1",
  "@vitejs/plugin-react": "^4.3.4",
  "babel-jest": "^29.7.0",
  "identity-obj-proxy": "^3.0.0",
  "jest": "^29.7.0",
  "jest-environment-jsdom": "^29.7.0",
  "vite": "^6.0.0",
  "chai": "^6.2.2",
  "mocha": "^12.0.0"
}
```

Vista del front

The image shows a dark-themed user interface for an 'Order Validation System'. At the top, there is a purple pill-shaped button labeled 'TDD WORKSHOP'. Below it, the title 'Order Validation System' is displayed in a large, bold, white font. Underneath the title, a subtitle in a smaller white font reads 'Enter order details to validate business rules'. The form consists of three input fields, each with a label above it: 'Product / Items' with a placeholder text 'e.g. Notebook, Pencil', 'Total Amount (\$)' with a placeholder '0', and 'Payment Method' with a dropdown menu showing 'Credit / Debit Card (tarjeta)' and a downward arrow. At the bottom of the form is a large, rounded purple button with the text 'Validate Order' in white.

TDD WORKSHOP

Order Validation System

Enter order details to validate business rules

Product / Items

e.g. Notebook, Pencil

Total Amount (\$)

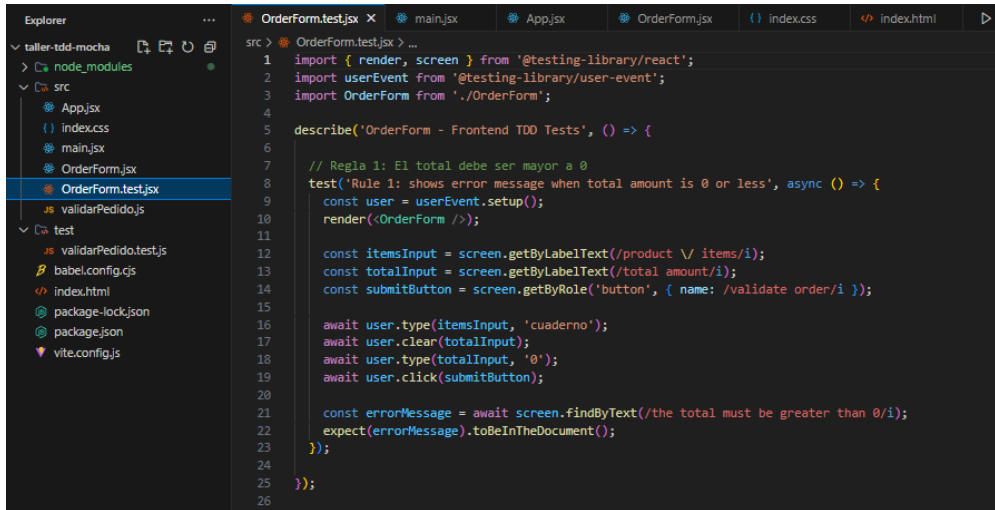
0

Payment Method

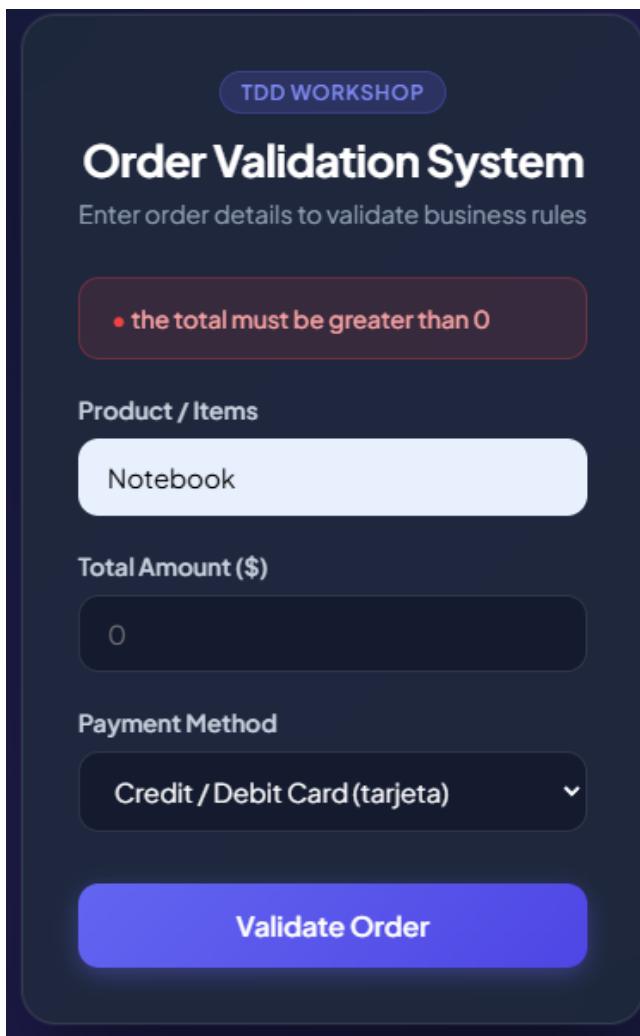
Credit / Debit Card (tarjeta) ▼

Validate Order

Regla 1



```
src > OrderForm.test.jsx > ...
1 import { render, screen } from '@testing-library/react';
2 import userEvent from '@testing-library/user-event';
3 import OrderForm from './OrderForm';
4
5 describe('OrderForm - Frontend TDD Tests', () => {
6
7   // Regla 1: El total debe ser mayor a 0
8   test('Rule 1: shows error message when total amount is 0 or less', async () => {
9     const user = userEvent.setup();
10    render(<OrderForm />);
11
12    const itemsInput = screen.getByLabelText(/product \\/ items/i);
13    const totalInput = screen.getByLabelText(/total amount/i);
14    const submitButton = screen.getByRole('button', { name: /validate order/i });
15
16    await user.type(itemsInput, 'cuaderno');
17    await user.clear(totalInput);
18    await user.type(totalInput, '0');
19    await user.click(submitButton);
20
21    const errorMessage = await screen.findByText(/the total must be greater than 0/i);
22    expect(errorMessage).toBeInTheDocument();
23  });
24
25 });
26
```



TDD WORKSHOP

Order Validation System

Enter order details to validate business rules

- the total must be greater than 0

Product / Items

Notebook

Total Amount (\$)

0

Payment Method

Credit / Debit Card (tarjeta) ▼

Validate Order

```
SyntaxError: Unexpected token 'export'

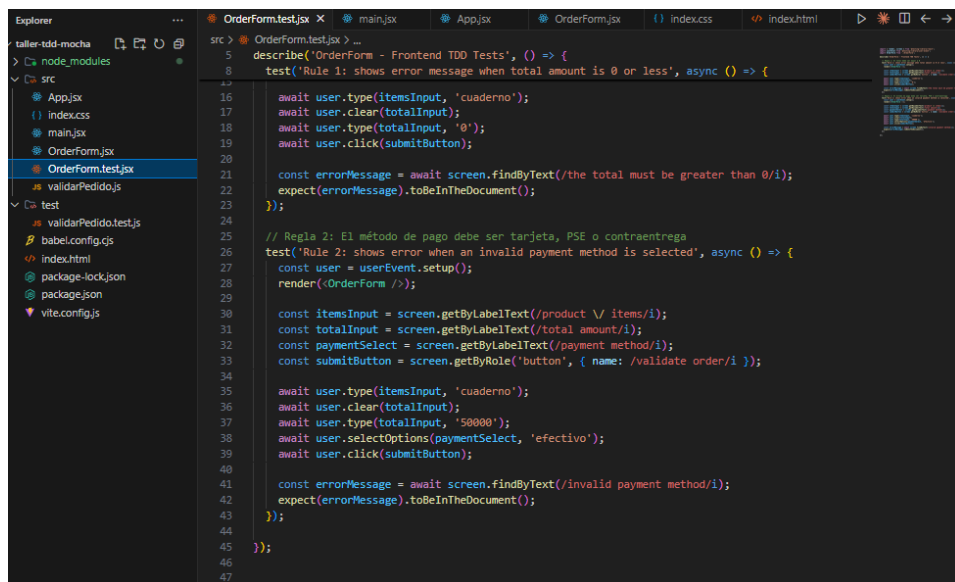
> 1 | const { expect } = require('chai');
    |                               ^
  2 | const validarPedido = require('../src/validarPedido');
  3 |
  4 | describe('validarPedido', () => {

    at Runtime.createScriptFromCode (node_modules/jest-runtime/build/index.js:1505:14)
    at Object.require (test/validarPedido.test.js:1:20)

PASS src/OrderForm.test.jsx

Test Suites: 1 failed, 1 passed, 2 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        5.247 s
Ran all test suites.
```

Regla 2



```
src > OrderForm.test.jsx > ...
5 describe('OrderForm - Frontend TDD Tests', () => {
8   test('Rule 1: shows error message when total amount is 0 or less', async () => {
16     await user.type(itemsInput, 'cuaderno');
17     await user.clear(totalInput);
18     await user.type(totalInput, '0');
19     await user.click(submitButton);
20
21     const errorMessage = await screen.findByText(/the total must be greater than 0/i);
22     expect(errorMessage).toBeInTheDocument();
23   });
24
25   // Regla 2: El método de pago debe ser tarjeta, PSE o contraentrega
26   test('Rule 2: shows error when an invalid payment method is selected', async () => {
27     const user = userEvent.setup();
28     render(<OrderForm />);
29
30     const itemsInput = screen.getByLabelText(/product \\\ items/i);
31     const totalInput = screen.getByLabelText(/total amount/i);
32     const paymentSelect = screen.getByLabelText(/payment method/i);
33     const submitButton = screen.getByRole('button', { name: /validate order/i });
34
35     await user.type(itemsInput, 'cuaderno');
36     await user.clear(totalInput);
37     await user.type(totalInput, '50000');
38     await user.selectOptions(paymentSelect, 'efectivo');
39     await user.click(submitButton);
40
41     const errorMessage = await screen.findByText(/invalid payment method/i);
42     expect(errorMessage).toBeInTheDocument();
43   });
44 });
45
46
47
```

```
PS C:\Users\JoseL\OneDrive\Escritorio\taller-tdd-mocha> npm test

> taller-front-tdd@1.0.0 test
> jest

PASS src/OrderForm.test.jsx
  OrderForm - Frontend TDD Tests
    ✓ Rule 1: shows error message when total amount is 0 or less (399 ms)
    ✓ Rule 2: shows error when an invalid payment method is selected (507 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        3.542 s, estimated 4 s
Ran all test suites.

PS C:\Users\JoseL\OneDrive\Escritorio\taller-tdd-mocha>
```

TDD WORKSHOP

Order Validation System

Enter order details to validate business rules

• Invalid payment method

Product / Items

Notebook

Total Amount (\$)

500

Payment Method

Cash (efectivo - Invalid) ▾

Validate Order

Regla 3

```
Explorer
  ✓ taller-tdd-mocha
    > node_modules
    > src
      App.jsx
      index.css
      main.jsx
      OrderForm.jsx
      OrderForm.test.jsx
      validarPedido.js
    > test
      validarPedido.test.js
      babel.config.js
      index.html
      package-lock.json
      package.json
      vite.config.js

src > OrderForm.test.jsx > ...
5 describe('OrderForm - Frontend TDD Tests', () => {
26   test('Rule 2: shows error when an invalid payment method is selected', async () => {
39     await user.click(submitButton);
40
41     const errorMessage = await screen.findByText(/invalid payment method/i);
42     expect(errorMessage).toBeInTheDocument();
43   });
44
45   // Regla 3: Si es contraentrega y supera 500000, no es válido
46   test('Rule 3: shows error when cash on delivery exceeds 500,000', async () => {
47     const user = userEvent.setup();
48     render(<OrderForm />);
49
50     const itemsInput = screen.getByLabelText(/product \\/ items/i);
51     const totalInput = screen.getByLabelText(/total amount/i);
52     const paymentSelect = screen.getByLabelText(/payment method/i);
53     const submitButton = screen.getByRole('button', { name: /validate order/i });
54
55     await user.type(itemsInput, 'cuaderno');
56     await user.clear(totalInput);
57     await user.type(totalInput, '600000');
58     await user.selectOptions(paymentSelect, 'contraentrega');
59     await user.click(submitButton);
60
61     const errorMessage = await screen.findByText(/cash on delivery is not available for orders
62     exceeding \$500,000\./i);
63     expect(errorMessage).toBeInTheDocument();
64   });
65 });
66
67
68
```

```
5 describe('OrderForm - Frontend TDD Tests', () => {
26   test('Rule 2: shows error when an invalid payment method is selected', async () => {
39     await user.click(submitbutton);
40
41     const errorMessage = await screen.findByText(/invalid payment method/i);
42     expect(errorMessage).toBeInTheDocument();
43   });
44
45   // Regla 3: Si es contraentrega y supera 500000, no es válido
46   test('Rule 3: shows error when cash on delivery exceeds 500,000', async () => {
47     const user = userEvent.setup();
48     render(<OrderForm />);
49
50     const itemInput = screen.getByLabelText(/product \d/i, item(i));
```

Problems Output Debug Console Terminal Ports

PS C:\Users\JoseL\OneDrive\Escritorio\taller-tdd-mocha> npm test

> taller-front-tdd@1.0.0 test

> jest

PASS src/OrderForm.test.jsx

OrderForm - Frontend TDD Tests

- ✓ Rule 1: shows error message when total amount is 0 or less (442 ms)
- ✓ Rule 2: shows error when an invalid payment method is selected (525 ms)
- ✓ Rule 3: shows error when cash on delivery exceeds 500,000 (518 ms)

Test Suites: 1 passed, 1 total

Tests: 3 passed, 3 total

Snapshots: 0 total

Time: 3.944 s

Ran all test suites.

PS C:\Users\JoseL\OneDrive\Escritorio\taller-tdd-mocha>

TDD WORKSHOP

Order Validation System

Enter order details to validate business rules

Cash on delivery is not available for orders exceeding \$500,000.

Product / Items

Notebook

Total Amount (\$)

500001

Payment Method

Cash on Delivery (contraentrega)

Validate Order

Camino FELIZ

```
src > OrderForm.test.jsx > ...
5 describe('OrderForm - Frontend TDD Tests', () => {
46   test('Rule 3: shows error when cash on delivery exceeds 500,000', async () => {
62     expect(errorMessage).toBeInTheDocument();
63   });
64
65   // Regla 4: Caso feliz (pedido completamente válido)
66   test('Rule 4: displays success message when order is completely valid', async () => {
67     const user = userEvent.setup();
68     render(<OrderForm />);
69
70     const itemsInput = screen.getByLabelText(/product \\/ items/i);
71     const totalInput = screen.getByLabelText(/total amount/i);
72     const paymentSelect = screen.getByLabelText(/payment method/i);
73     const submitButton = screen.getByRole('button', { name: /validate order/i });
74
75     await user.type(itemsInput, 'cuaderno, lapicero');
76     await user.clear(totalInput);
77     await user.type(totalInput, '45000');
78     await user.selectOptions(paymentSelect, 'tarjeta');
79     await user.click(submitButton);
80
81     const successMessage = await screen.findByRole('status');
82     expect(successMessage).toHaveTextContent(/order placed successfully!/i);
83     expect(screen.queryByRole('alert')).not.toBeInTheDocument();
84   });
85
86 });
87
88
89
90
```

```
PS C:\Users\JoseL\OneDrive\Escritorio\taller-tdd-mocha> npm test

> taller-front-tdd@1.0.0 test
> jest

PASS src/OrderForm.test.jsx
  OrderForm - Frontend TDD Tests
    ✓ Rule 1: shows error message when total amount is 0 or less (438 ms)
    ✓ Rule 2: shows error when an invalid payment method is selected (528 ms)
    ✓ Rule 3: shows error when cash on delivery exceeds 500,000 (514 ms)
    ✓ Rule 4: displays success message when order is completely valid (625 ms)

Test Suites: 1 passed, 1 total
Tests: 4 passed, 4 total
Snapshots: 0 total
Time: 4.759 s
Ran all test suites.
PS C:\Users\JoseL\OneDrive\Escritorio\taller-tdd-mocha>
```

Link del repositorio: <https://github.com/V0ru/taller-tdd-mocha>

Para ejecutar las pruebas al backend escribir en la terminal : `npm run test:backend`

Para ejecutar las del front escribir en el terminal: `npm test`